

预训练目标与数据构造实验

第054单元 独立操作与验收

目标是在同一组文档上构造AR与简化MLM，验证标签移位、独立文档packing、loss归约和数据去重。编码解码只做手算信息边界图。本实验是未训练模型的结构检查，不比较目标的泛化表现。直接先修019、021、053，建议先阅读lecture.pdf第一至五节。

一 安装和重跑

本目录包含数据生成器、原始11条文本、规范化后10份唯一文档、15词元闭合词表、所有张量构造及完整结果。无需联网下载语料或模型，无GPU、付费API要求。主要环境Python3.12、torch2.7.1+cpu、numpy2.3.5、matplotlib3.10.8，Linux单线程。建议环境与命令：

```
conda env create -f environment.yml
conda activate dl054
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1
python test_experiment.py
python -O test_experiment.py
python experiment.py --output replay
python make_figures.py --results replay/results.json --output replay/figures
jupyter lab experiment.ipynb
```

已有Python3.12也可用venv及pip install -r requirements.txt。torch的CPU轮子选择见PyTorch官方安装说明。图与完整Notebook需Noto Sans CJK；默认路径/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc。Debian/Ubuntu安装fonts-noto-cjk；其他平台从Google Noto官方发行安装，并把DL_CJK_FONT设为实际ttc/otf路径。缺字体明确报错；数值实验不需字体。

Run All重算数据、损失、测试和6张图，写notebook_replay。阅读随包已执行Notebook不需重建；create_notebook.py仅用于生成空Notebook。python execute_notebook.py experiment.ipynb使用新进程内的IPython in-process内核核验顺序执行；未测试Jupyter浏览器UI、socket、GPU或其他操作系统。

运行数值核心通常几秒内，学习及手算建议2小时。不要为了更低loss修改固定目标或删除失败对照。新实验写新目录，不覆盖随包outputs作为比较基准。

二 输入和目标的五个练习

练习1 打印预测对

给正文A=[蓝,猫]、B=[红]，分别写出AR input、label、position、有效目标数，再拼成一个packed样本。解释为什么BOS不作此处的预测目标、EOS参加loss。写出“在数据函数移位一次，模型内部又移位一次”会预测什么。

练习2 区分三种mask

给正文[蓝,猫,追,红,球]，选正文位置1、3、5（从1计正文），全部替换MASK。写出带BOS/EOS的输入与标签。画出attention可见性，再标记loss有效位置。说明未选中的“猫”能否被其他query读取、其输出是否计loss、其嵌入能否收到梯度。

练习3 手算NLL与两次更新

三个类别logits分别为位置0的($\ln 2, 0, 0$)、位置1的($0, \ln 3, 0$)、忽略位置2的($9, -4, 1$)，标签为[0,1,IGNORE]。计算softmax、总NLL、有效数、平均NLL、所有logit梯度，学习率0.2更新两次，每一步重新计算概率。写出如果logits来自 $Z=HW+b$ ，梯度如何继续传回W、b、H。用nll_ledger核对。

练习4 两个微批的归约

微批A有2个有效token，总NLL0.2；微批B有6个，总NLL12。比较sum/count与平均两个mean。说明如何在两次backward后执行一次step，得到按8个token平均的总梯度。如果两个batch有效数相同，哪种错误会暂时被掩盖？

练习5 编码解码

源为[蓝,MASK,球]，目标为[猫,追,EOS]。写decoder input和label，画encoder self-attention、decoder self-attention、cross-attention三个mask的允许范围。解释为什么目标未移位会让当前词元直接泄漏，为什么把完整原目标放入encoder也可能使去噪任务失效。

三 运行结构干预与数据审计

练习6 Packing等价性

阅读ar_padded、ar_packed。检查分块因果mask和位置重置；从padded输出中按valid选取 logits，与packed对应输出逐元素比较。正式阈值 $2e-12$ ，测试还对齐每个模型参数的梯度。

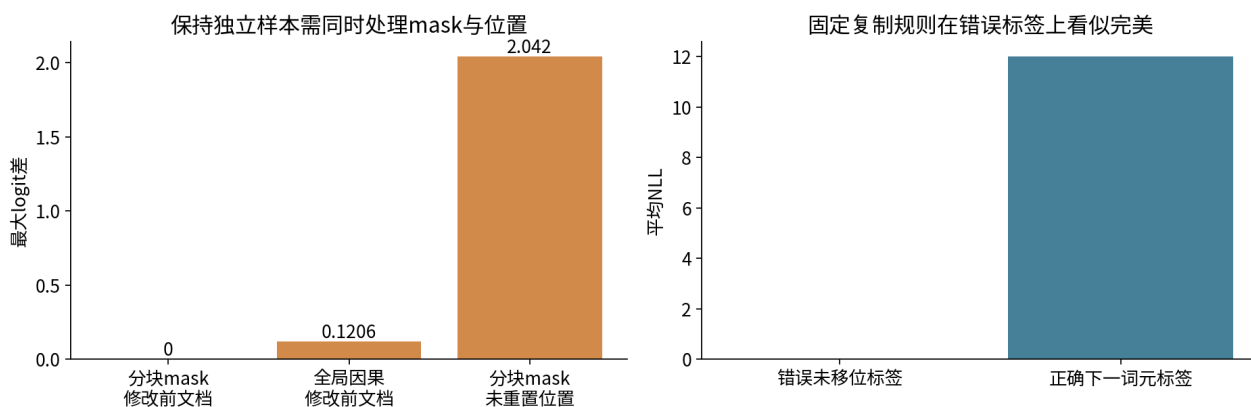
依次注入两个错误：只用全局下三角mask；正确分块但position使用整条长行的连续编号。分别改变第一文档并观察后续文档、比较逐文档基准。必须一次只改变一个条件。记录没有变化和出现变化的结果，不能把非零反例当失败而删掉。

练习7 标签错位诊断

复制规则给当前位置input token的logit设6，其他类设-6。在错误未移位标签与正确AR标签下分别计算loss。解释为什么因果mask正确也无法挽救错误标签。不要把此规则包装成“训练成功的语言模型”。

练习8 去重与划分

查看data/corpus.json。找出d00和d08的关系，复算normalize和SHA256。确认训练6、验证2、测试2，没有规范化文本哈希交叉。解释为什么这不证明没有语义近重复，为什么词元重叠本身不是泄漏。说明词表是预先声明还是从数据拟合，若改成BPE需要怎样防止评估信息进入训练流程。



审计图 正确packing需要文档边界和位置协议同时一致。右侧复制诊断故意保留接近零的错误loss，防止只看曲线就误判。

四 验收与故障排查

必须提交：输入/标签/位置/三种mask示例、NLL完整推导和两次更新、8个练习回答、7类unittest结果、packing前向与所有参数梯度对齐记录、重复映射与split计数、全部错误对照。Notebook的6幅图应是Run All本次生成的真实image/png输出。

固定协议应得到AR有效数34、MLM有效数10。AR与packing平均NLL均约2.742892754；MLM约2.603452297但不能据此排名。有效logits最大packing误差约4.44e-16；阻断后跨文档干预差0；全局因果泄漏差约0.120594；不重置位置差约2.041716。不同软件内核可能有微小浮点差，结构测试使用合理容差。

常见问题：

- loss非常低：先打印当前input和label是否相同，再查是否重复移位或原文副本泄漏。
- shape匹配但loss不对：检查类别轴。我们的logits是 $B \times L \times V$ ，官方CrossEntropyLoss直接接口通常需要展平为 $(BL) \times V$ 或转成 $B \times V \times L$ 。[1]
- 全padding或MLM零选中：平均loss分母为0。程序明确拒绝无有效目标；不能静默返回0冒充成功训练。
- PAD仍影响真实输出：确认真实query读不到padding key。只把label设IGNORE不会切断上下文路径。
- packed输出不同：检查位置重置、segment边界、dropout关闭、参数相同及是否额外截断。
- IGNORE索引报错：先筛选labels!= -100，再gather；不要把IGNORE送入Embedding。
- 跨批验证loss不稳定：保留总loss与有效数，最后统一相除，而非简单平均batch mean。

边界：空文档、特殊符号混入正文、词表外ID明确拒绝；本单元不做语义去重、流式截断、分布式归约、半精度或实际语言能力评测。position编号必须在0至127。长文档需要另行定义窗口协议，不能无声截断。

可选PDF重建按build-tools/README.md，在单元目录安装Python构建依赖、`npm ci --prefix .build-deps`并运行`python build_pdf.py lecture.md`等。需要Node、WeasyPrint系统依赖、Noto Sans/Serif CJK。构建器只用于可信源码。

[1] PyTorch2.7 CrossEntropyLoss: <https://docs.pytorch.org/docs/2.7/generated/torch.nn.CrossEntropyLoss.html>。原论文及其他一手来源见sources.md。本单元数据许可见DATA_LICENSE.md。